Assignment #:02

Course: Visual Programming Lab

Submitted By:

Samra Ramzan

232201009

CS-5A

Submitted To:

Sir Uzair Hassan

Department of Computer Science

Kahuta Institute of Space and Technology

March 29, 2026

Contents

1 Project Introduction **3**

1.1 Problem Statement 3

1.2 Objective 3

2 System Design **3**

2.1 Architecture Diagram 3

2.2 Workflow 4

3 Tools & Technologies **4**

4 Code Snippets **4**

4.1 1. User Input Handling 4

4.2 2. API Integration (MyMemory) 5

4.3 3. Button Click Event 5

4.4 4. Error Handling 5

5 Application Preview **6**

5.1 Form Home 6

5.2 Form Menu 7

5.3 Form Translate 7

5.4 Form History 8

5.5 Form About 8

6 Testing **9**

7 Results & Discussion **9**

8 Future Improvements **9**

9 Docker Implementation **10**

9.1 Dockerfile Screenshot 10

9.2 Dockerfile Code 10

9.3 Build Command 10

9.4 Run Command 10
9.5 Container Screenshot 11

1 Project Introduction

1.1 Problem Statement

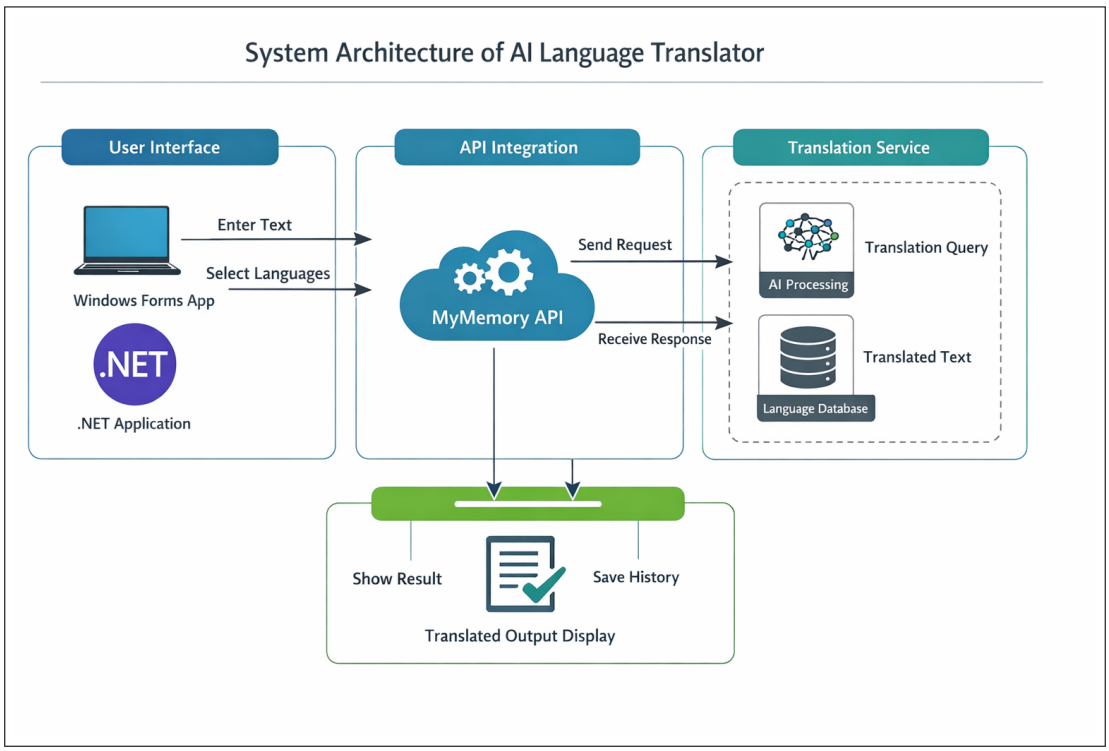
Communication between different languages is difficult for users. There is a need for a system that can easily translate text into different languages.

1.2 Objective

- Develop AI-based Language Translator
- Provide fast and accurate translation
- Simple and user-friendly interface

2 System Design

2.1 Architecture Diagram



figureSystem Architecture

2.2 Workflow

1. User enters text
2. Selects source and target language
3. Clicks translate button
4. MyMemory API processes request
5. Output displayed on screen

3 Tools & Technologies

- .NET Framework / .NET 7
- C#
- Windows Forms
- MyMemory Translation API
- Docker

4 Code Snippets

4.1 1. User Input Handling

```
// INPUT
txtInput = new TextBox();
txtInput.Multiline = true;
txtInput.Size = new Size(220, 80);
txtInput.Location = new Point(30, 110);
txtInput.Font = new Font("Segoe UI", 10f);
txtInput.BorderStyle = BorderStyle.None;
txtInput.BackColor = Color.FromArgb(240, 250, 249);

Panel inputPanel = CreateRoundedPanel(txtInput);
```

Explanation: This code takes input from the user through the textbox.

4.2 2. API Integration (MyMemory)

```
using (HttpClient client = new HttpClient())
{
    string encodedInput = Uri.EscapeDataString(input);
    string url = $"https://api.mymemory.translated.net/get?q={
        encodedInput}&langpair={fromLang}|{toLang}";

    HttpResponseMessage response = await client.GetAsync(url);
    string jsonResponse = await response.Content.ReadAsStringAsync();

    if (!response.IsSuccessStatusCode)
        throw new Exception($"API Error: {response.StatusCode}");
```

Explanation: This code sends request to MyMemory API and gets translated text.

4.3 3. Button Click Event

```
// TRANSLATE BUTTON
btnTranslate = new Button();
btnTranslate.Text = "Translate";
btnTranslate.Size = new Size(200, 45);
btnTranslate.Location = new Point(40, 260);
btnTranslate.BackColor = teal;
btnTranslate.ForeColor = white;

btnTranslate.Click += async (s, e) => await TranslateTextAsync();
```

Explanation: This function handles translation when button is clicked.

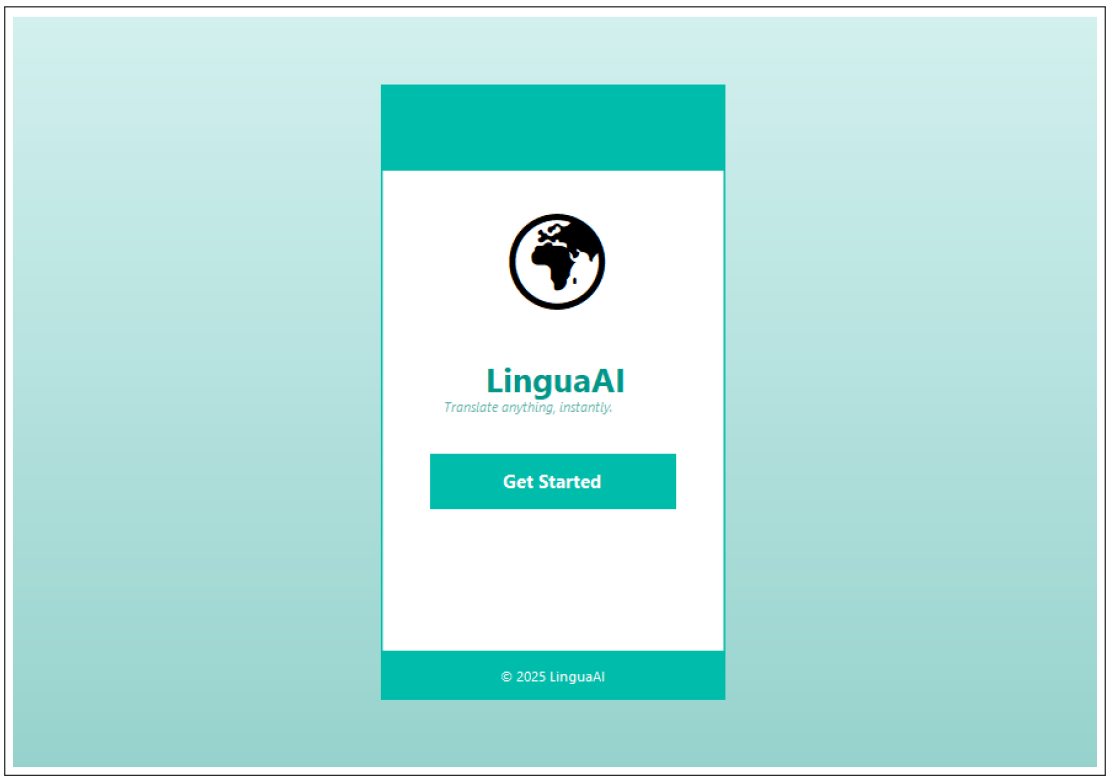
4.4 4. Error Handling

```
catch (HttpRequestException ex)
{
    MessageBox.Show($"Network error: {ex.Message}\n\nPlease check your
        internet connection.",
        "Translation Failed", MessageBoxButtons.OK, MessageBoxIcon.
            Error);
}
```

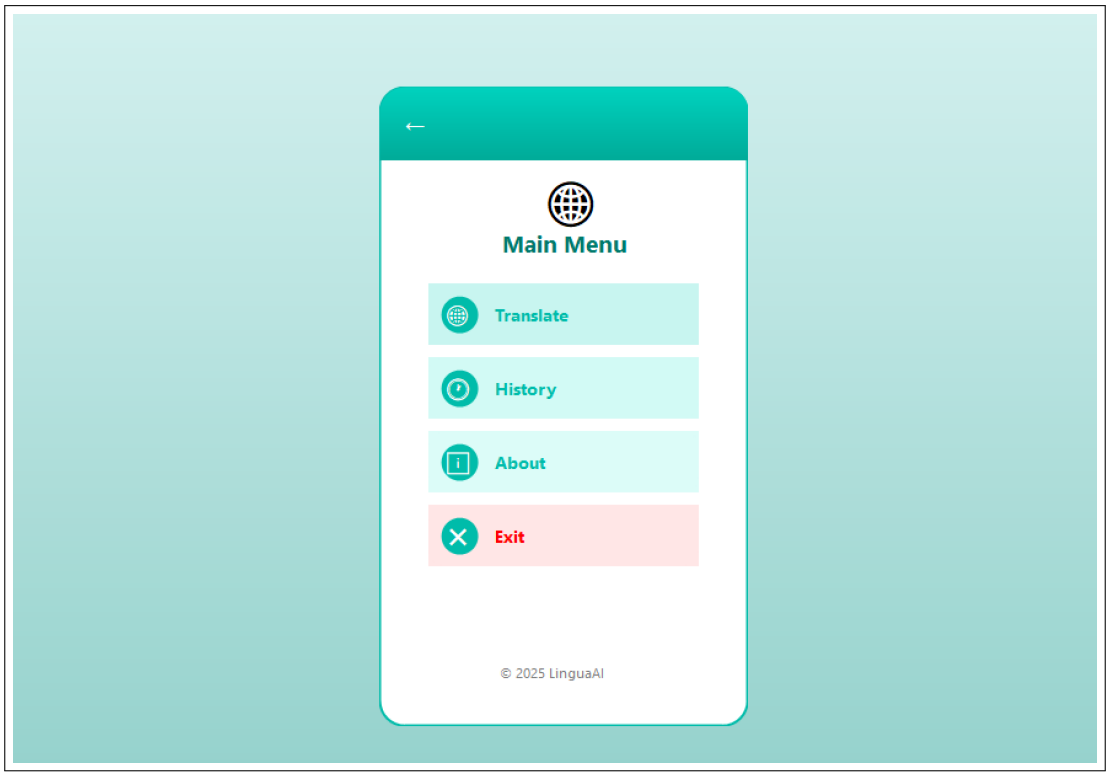
Explanation: Prevents application crash and handles errors.

5 Application Preview

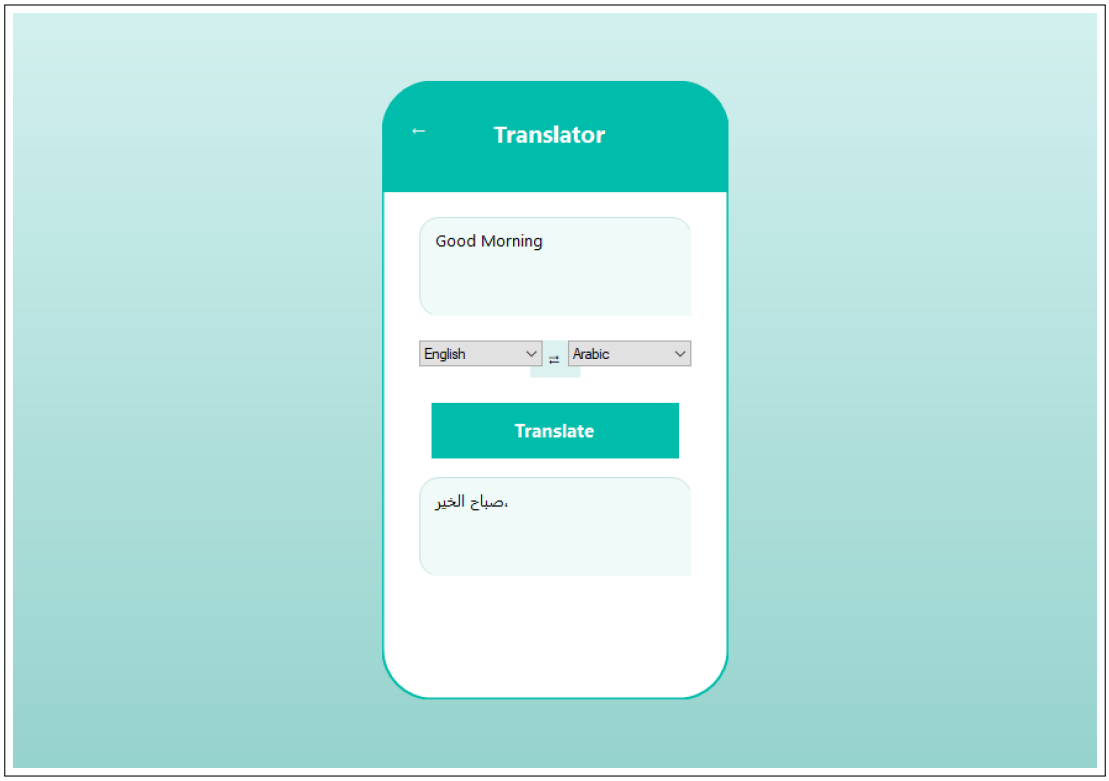
5.1 Form Home



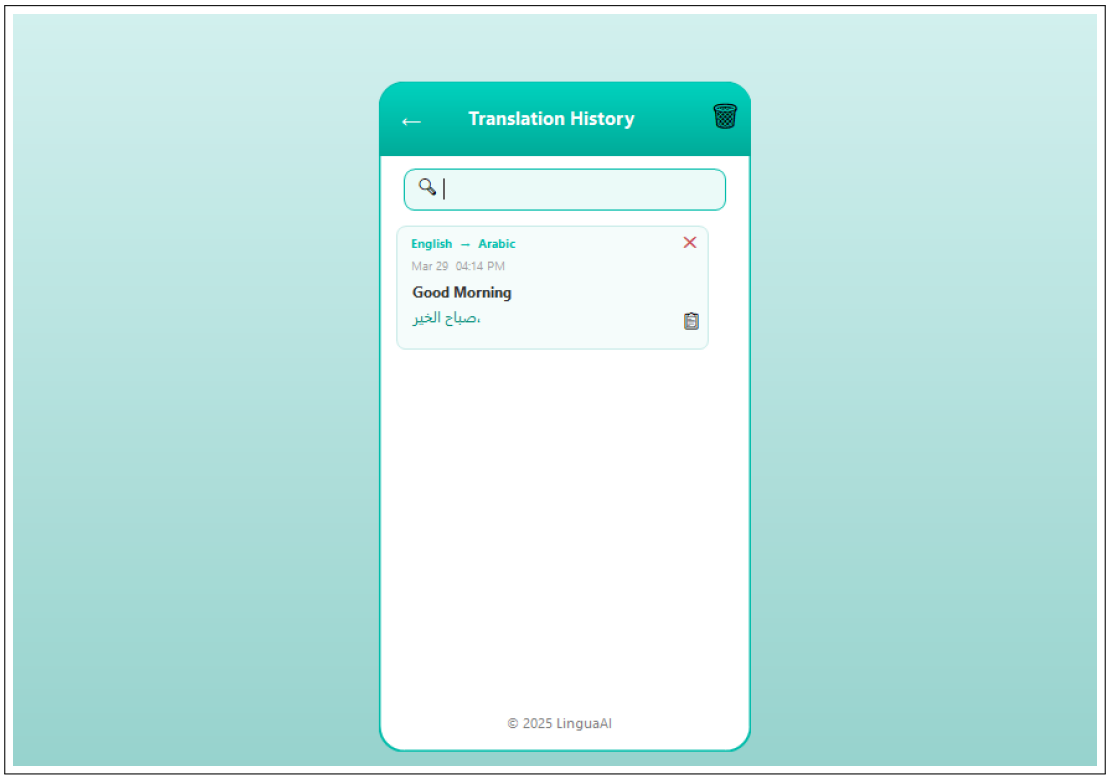
5.2 Form Menu



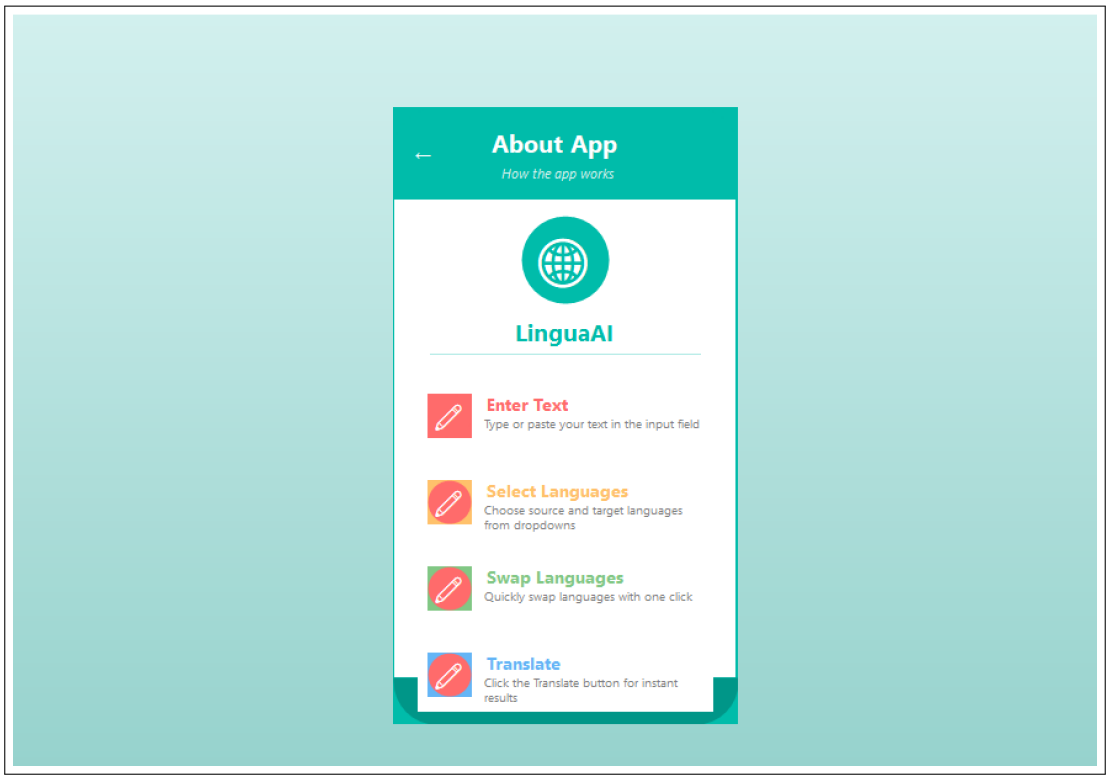
5.3 Form Translate



5.4 Form History



5.5 Form About



6 Testing

Input	Language	Output
Hello	English → Urdu	
Good Night	English → French	Bonne Nuit
How are you	English → Arabic	

7 Results & Discussion

What Worked Well:

- Accurate translations
- Fast response
- Easy to use interface

Limitations:

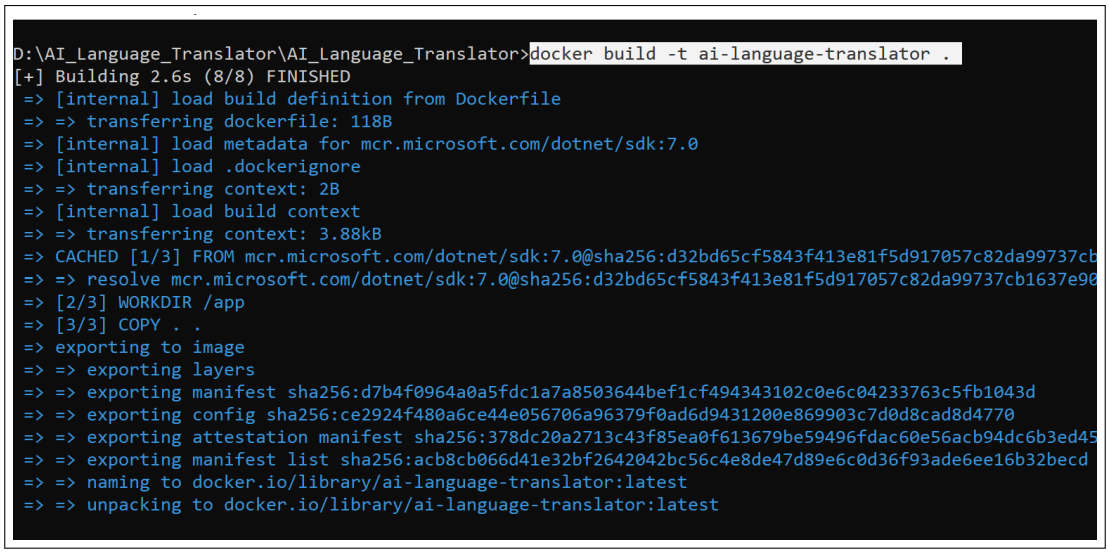
- Requires internet connection
- Limited free API usage

8 Future Improvements

- Voice input feature
- Offline translation
- More language options

9 Docker Implementation

9.1 Dockerfile Screenshot



```
D:\AI_Language_Translator\AI_Language_Translator>docker build -t ai-language-translator .
[+] Building 2.6s (8/8) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 118B
 => [internal] load metadata for mcr.microsoft.com/dotnet/sdk:7.0
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [internal] load build context
 => => transferring context: 3.88kB
 => CACHED [1/3] FROM mcr.microsoft.com/dotnet/sdk:7.0@sha256:d32bd65cf5843f413e81f5d917057c82da99737cb
 => => resolve mcr.microsoft.com/dotnet/sdk:7.0@sha256:d32bd65cf5843f413e81f5d917057c82da99737cb1637e90
 => [2/3] WORKDIR /app
 => [3/3] COPY . .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:d7b4f0964a0a5fdc1a7a8503644bef1cf494343102c0e6c04233763c5fb1043d
 => => exporting config sha256:ce2924f480a6ce44e056706a96379f0ad6d9431200e869903c7d0d8cad8d4770
 => => exporting attestation manifest sha256:378dc20a2713c43f85ea0f613679be59496fdac60e56acb94dc6b3ed45
 => => exporting manifest list sha256:acb8cb066d41e32bf2642042bc56c4e8de47d89e6c0d36f93ade6ee16b32becd
 => => naming to docker.io/library/ai-language-translator:latest
 => => unpacking to docker.io/library/ai-language-translator:latest
```

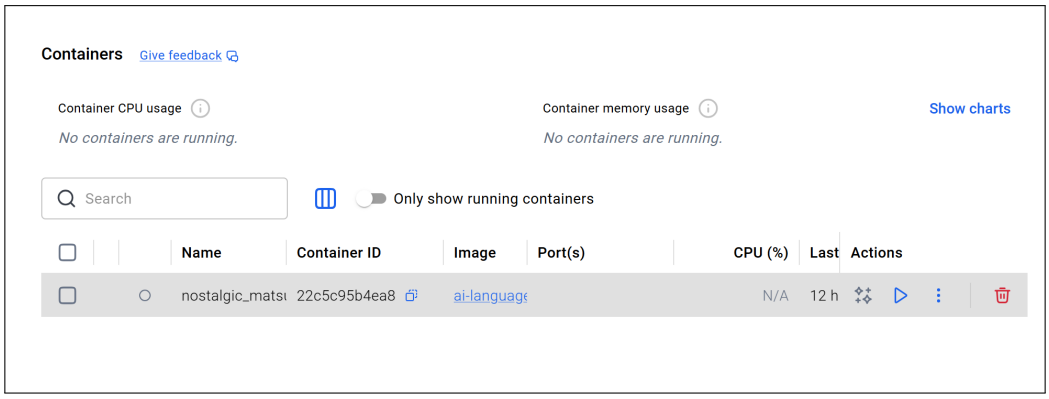
9.2 Dockerfile Code

```
FROM mcr.microsoft.com/dotnet/sdk:7.0
WORKDIR /app
COPY . .
RUN dotnet build
CMD ["dotnet", "run"]
```

9.3 Build Command

```
docker build -t ai-language-translator .
```

9.4 Run Command

```
docker run ai-language-translator
```

9.5 Container Screenshot